CENG-322 Mobile Application Development - Deliverable 5

Team Name: IoT Innovators

Project Name: Smart Farm

Team Number: 8

Student's names and IDs:

- Chau Vi Cuong Nguyen – N01634611
- Hoang Tran – N01613506
- Tuan Anh Pham – N01238459
- Truong Anh Nguyen – N01639800

Table of Contents

Member Info and Participation 4

Project Description 4

List number of commits 4

Sprint 5 Goals: 5

3 Tasks that are related to addressing technical debt 5

Detailed work that has been completed 5

Complete Scrum Dashboard 6

Screenshots showing: 9

A. AlertDialog 9

B. Snackbar 10

C. Toast 11

D. Notification 12

Screenshot showing the fab button and how the functionality fits into your app. 13

Document what feature will work off-line. Should be meaningful feature. Will be tested, when in Airplane mode. Provide screenshot 16

Exception Handler 20

Post-Mortem, Project Review Meeting, document the below: 21

A. Performance review of the project 21

Cost 21

Schedule 21

Quality 21

B. Time Management Evaluate 22

C. Issues with the quality or compromises along the way 22

D. Lessons learned, mistakes, and areas of improvement. 22

E. Metting attendances 23

F. Technical Debt solutions 23

Coding standards and their usage 24

Standard 1: Single Responsibility Principle (SRP) 24

Standard 2: Using Modern Android APIs 26

Potential security vulnerability and future approach 28

Screen Tracking Analytics 29

Test Cases Creation 30
Screenshot showing Espresso test cases are passing..... 32
Screenshots showing the data in your DB, sensors data (at least 3 different sensors), and login data (including Google login) screenshot showing the data stored into the 33
DB from Feedback. Must have at least 3 different entries..... 33
Simulation Script Documentation 37

Member Info and Participation

Name	Student ID	GitHub ID	Signature	Effort
Chau Vi Cuong Nguyen	N01634611	ChauViCuongNguyen4611	CVCN	100
Hoang Tran	N01613506	HoangTranTran3506	HT	100
Tuan Anh Pham	N01238459	TuanAnhPham8459	TA	90
Truong An Nguyen	N01639800	TruongAnNguyen9800	TAN	90

Project Description

The Smart Farm IoT application aims to create a comprehensive mobile solution for modern farming management. The final vision is to provide farmers with real-time monitoring and control of IoT-enabled farm equipment, including temperature monitors, automated irrigation systems, and crop health trackers.

List number of commits

Team member	Chau Vi Cuong Nguyen	Hoang Tran	Tuan Anh Pham	Truong An Nguyen
August 2	5			2
August 3	2			
August 4	2			
August 5	3		5	5
August 6		3	5	4
August 7	4			4
August 8	6	8	6	3
August 9	7	10	4	2
Total	29	21	20	20

Sprint 5 Goals:

1. Complete all remaining functionality to make the app production-ready\
2. Implement offline mode with meaningful functionality (cached sensor data)
3. Add Floating Action Button (FAB) with proper functionality on home screen
4. Create Python script to simulate sensor data writing to Firebase DB
5. Update UX/UI of all screens (ensure there's no hardcode, errors)

3 Tasks that are related to addressing technical debt

1. Remove Hardcoded String and Use Resources

- Description: Replace all hardcoded strings (e.g., "Login", "Email", "Password", "Welcome") with references to strings.xml to enable localization and easier maintenance
- Why Technical Debt: Hardcoded strings made internationalization (French/English) impossible and caused duplication across multiple XML files. Any text change required updating multiple files

2. Refactor LoginActivity – Separate Validation Logic

- Description: Extract validation logic from LoginActivity into a separate LoginValidator class to follow Single Responsibility Principle (SRP)
- Why Technical Debt: LoginActivity handles both UI display and validation logic (over 100 lines), making it difficult to test and maintain. Any change to validation required touching UI code

1. Removed unused Imports and Dead Code

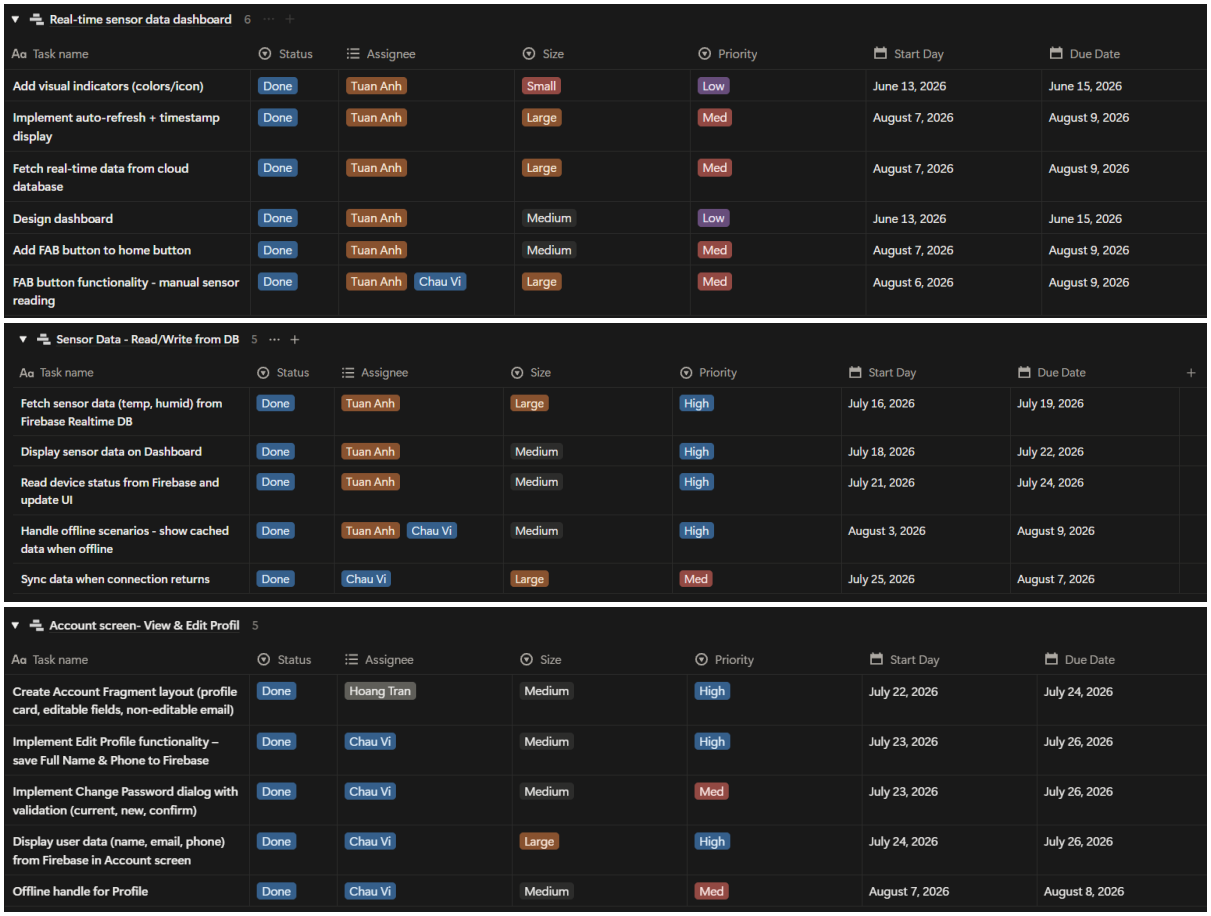
- Description: Clean up unused imports, commented-out code, and dead variables across all Java classes.
- Why Technical Debt: Dead code increases maintenance overhead, makes the codebase harder to understand, and slows down compilation

Detailed work that has been completed

Name	Sprint 5 Worked Completed
Chau Vi Cuong Nguyen	- Implemented offline mode with cached sensor data using Room DB - Removed unused imports and dead code (Technical Debt) - Integrated sensor data fetch from Firebase with ValueEventListener
Hoang Tran	- Completed UX/UI updates across all screens for polished, professional look - Wrote Espresso test class with 5+ test cases to test the UI of Login Activity

	- Wrote Junit test class (AlertViewTest)
Tuan Anh Pham	- Added FAB button to home screen with manual - Created analytics table with user data (Last 24 hrs, Last 7 days) sensor data entry functionality - Update function for Analytic Graph page (add function to custom tab, remember user last device)
Truong An Nguyen	- Wrote Junit test classes (Login & Sign up Validate) - Update function for customer to add 0.5 star in feedback - Wrote Robolectric test (for Feedback fragment)

Complete Scrum Dashboard



Real-time sensor data dashboard 6

Task name	Status	Assignee	Size	Priority	Start Day	Due Date
Add visual indicators (colors/icon)	Done	Tuan Anh	Small	Low	June 13, 2026	June 15, 2026
Implement auto-refresh + timestamp display	Done	Tuan Anh	Large	Med	August 7, 2026	August 9, 2026
Fetch real-time data from cloud database	Done	Tuan Anh	Large	Med	August 7, 2026	August 9, 2026
Design dashboard	Done	Tuan Anh	Medium	Low	June 13, 2026	June 15, 2026
Add FAB button to home button	Done	Tuan Anh	Medium	Med	August 7, 2026	August 9, 2026
FAB button functionality - manual sensor reading	Done	Tuan Anh, Chau Vi	Large	Med	August 6, 2026	August 9, 2026

Sensor Data - Read/Write from DB 5

Task name	Status	Assignee	Size	Priority	Start Day	Due Date
Fetch sensor data (temp, humid) from Firebase Realtime DB	Done	Tuan Anh	Large	High	July 16, 2026	July 19, 2026
Display sensor data on Dashboard	Done	Tuan Anh	Medium	High	July 18, 2026	July 22, 2026
Read device status from Firebase and update UI	Done	Tuan Anh	Medium	High	July 21, 2026	July 24, 2026
Handle offline scenarios - show cached data when offline	Done	Tuan Anh, Chau Vi	Medium	High	August 3, 2026	August 9, 2026
Sync data when connection returns	Done	Chau Vi	Large	Med	July 25, 2026	August 7, 2026

Account screen- View & Edit Profil 5

Task name	Status	Assignee	Size	Priority	Start Day	Due Date
Create Account Fragment layout (profile card, editable fields, non-editable email)	Done	Hoang Tran	Medium	High	July 22, 2026	July 24, 2026
Implement Edit Profile functionality – save Full Name & Phone to Firebase	Done	Chau Vi	Medium	High	July 23, 2026	July 26, 2026
Implement Change Password dialog with validation (current, new, confirm)	Done	Chau Vi	Medium	Med	July 23, 2026	July 26, 2026
Display user data (name, email, phone) from Firebase in Account screen	Done	Chau Vi	Large	High	July 24, 2026	July 26, 2026
Offline handle for Profile	Done	Chau Vi	Medium	Med	August 7, 2026	August 8, 2026

▼ Help screen 3

Task name	Status	Assignee	Size	Priority	Start Day	Due Date
Design Help screen	Done	Hoang Tran	Small	Med	August 7, 2026	August 9, 2026
Show information about alerts and dashboard	Done	Hoang Tran	Small	Low	August 7, 2026	August 9, 2026
Contact Support Page (navigate user to email when they need assistance)	Done	Hoang Tran	Medium	Med	August 8, 2026	August 9, 2026

▼ Historical analytics 5

Task name	Status	Assignee	Size	Priority	Start Day	Due Date
Display graphs with real time values	Done	Tuan Anh	Large	Med	August 5, 2026	August 9, 2026
Display average, minimum, and maximum values	Done	Tuan Anh	Medium	Med	August 5, 2026	August 9, 2026
Generate sensors data trend graphs	Done	Tuan Anh	Small	Med	July 23, 2026	July 25, 2026
Design analytics screen	Done	Tuan Anh	Medium	Med	June 18, 2026	June 20, 2026
Refresh page automatically after a period of time	Done	Tuan Anh	Medium	Med	August 5, 2026	August 9, 2026

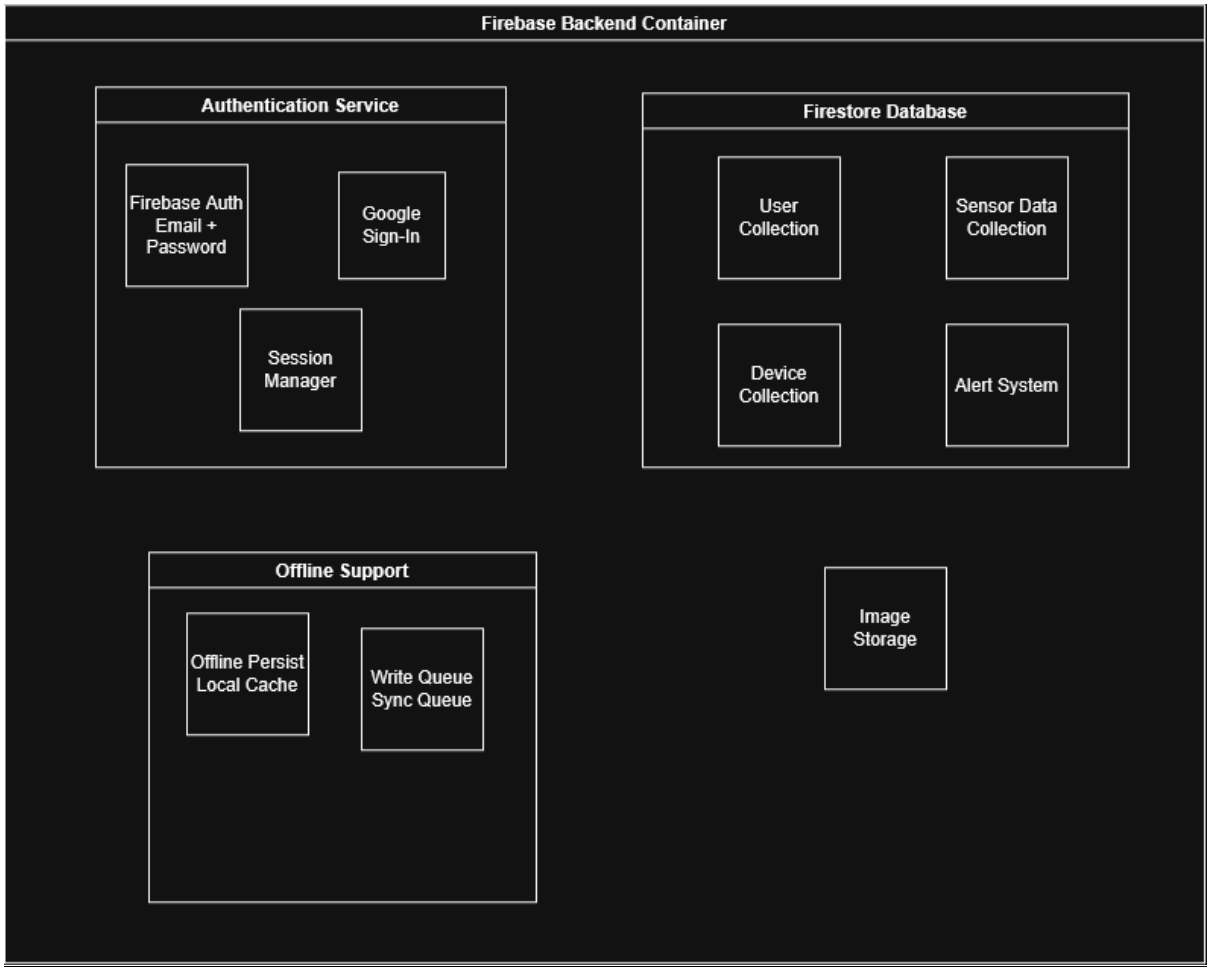
Projects

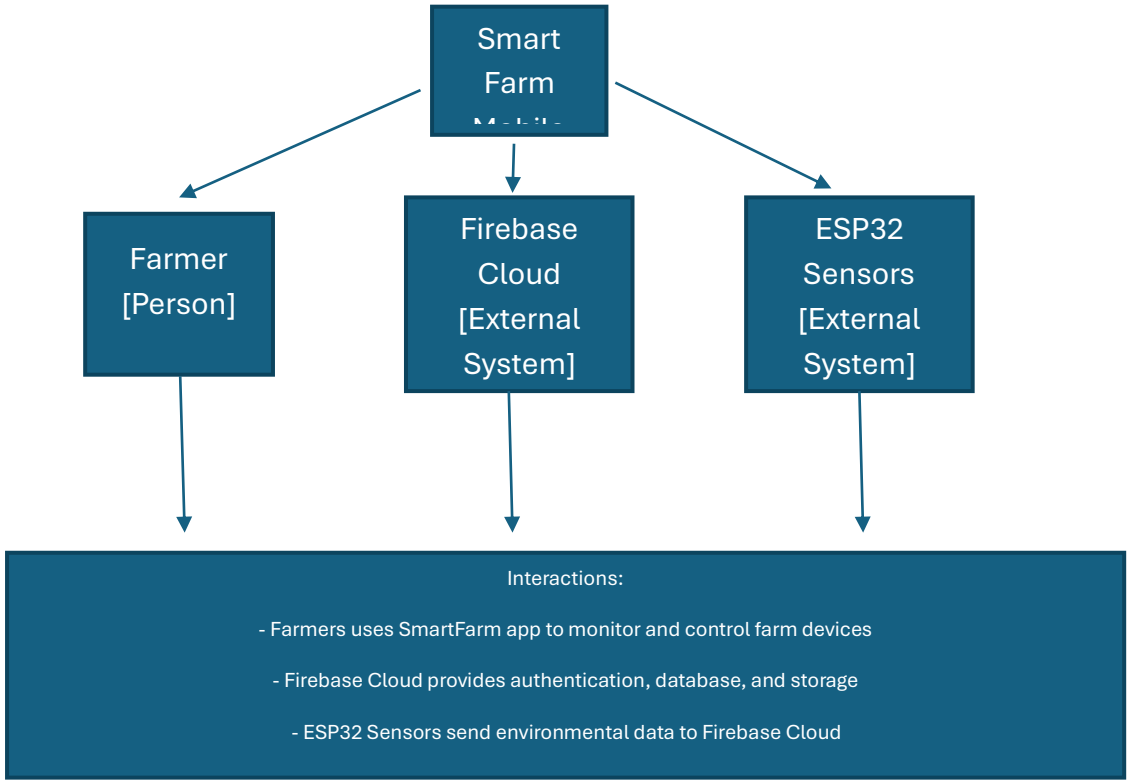
In progress | All Projects | Load per person

New

Name	Status	Project Timeline	Assignee
Help screen	Done	August 2, 2026 → August 9, 2026	Hoang Tran
Alert notification	Done	August 1, 2026 → August 9, 2026	Hoang Tran, Chau Vi
Historical analytics	Done	July 25, 2026 → August 8, 2026	Tuan Anh
Account screen- View & Edit Profil	Done	July 22, 2026 → August 8, 2026	Hoang Tran, Chau Vi
Real-time sensor data dashboard	Done	July 20, 2026 → August 8, 2026	Tuan Anh, Chau Vi
Sensor Data - Read/Write from DB	Done	July 13, 2026 → August 8, 2026	Chau Vi, Tuan Anh
Customer Feedback	Done	July 6, 2026 → July 26, 2026	Hoang Tran, Truong An
Toolbar menu	Done	June 10, 2026 → June 13, 2026	Truong An
User authentication	Done	June 8, 2026 → June 11, 2026	Chau Vi, Hoang Tran
Settings screen	Done	June 8, 2026 → June 12, 2026	Tuan Anh, Chau Vi

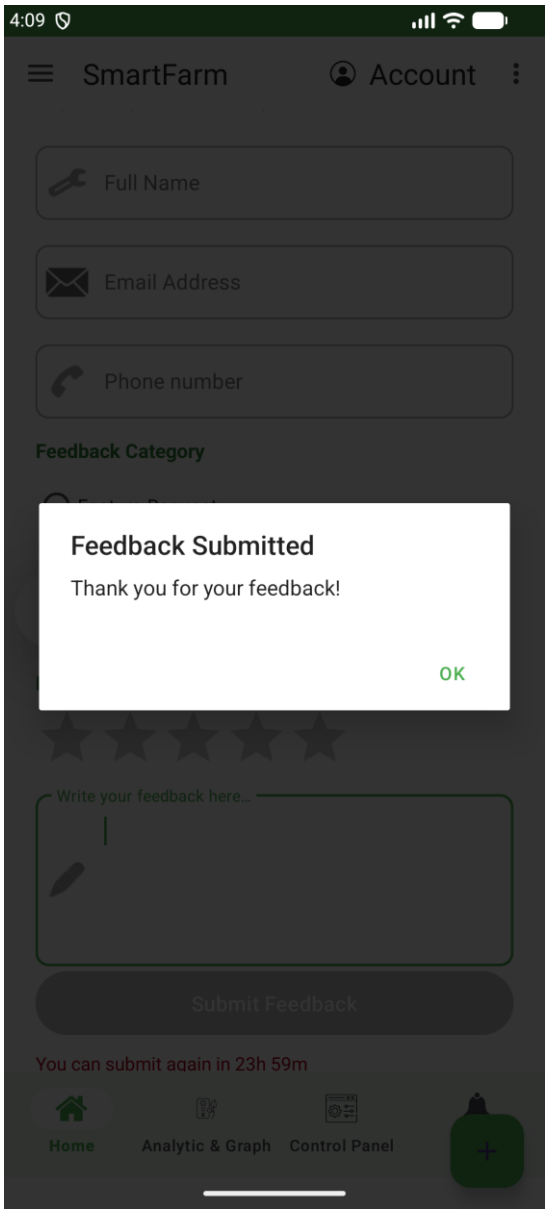
Using C4 Model, draw “Component Diagram”. Choose two of the containers in your system.


Firebase Backend Container
Authentication Service
Firebase Auth Email + Password
Google Sign-In
Session Manager
Firestore Database
User Collection
Sensor Data Collection
Device Collection
Alert System
Offline Support
Offline Persist Local Cache
Write Queue Sync Queue
Image Storage

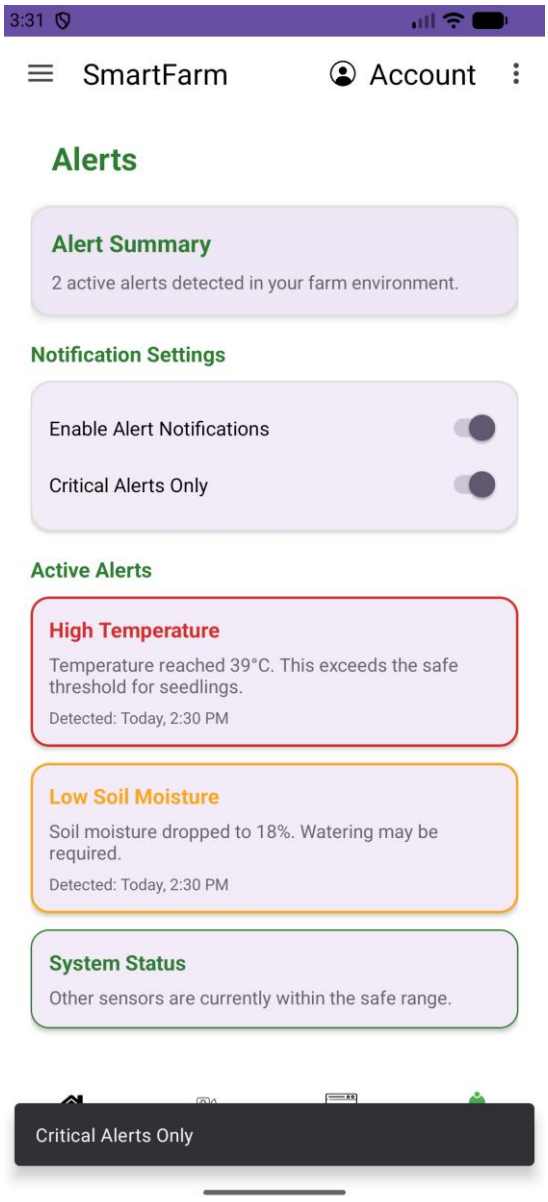

Smart Farm
Farmer [Person]
Firebase Cloud [External System]
ESP32 Sensors [External System]
Interactions:
- Farmers uses SmartFarm app to monitor and control farm devices
- Firebase Cloud provides authentication, database, and storage
- ESP32 Sensors send environmental data to Firebase Cloud

Screenshots showing:

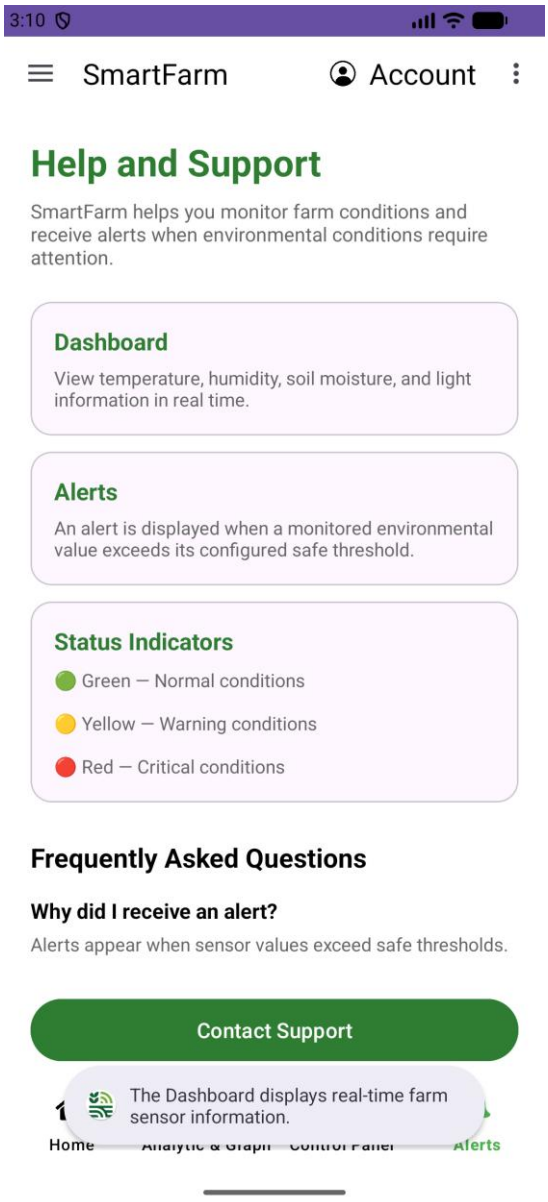
A. AlertDialog



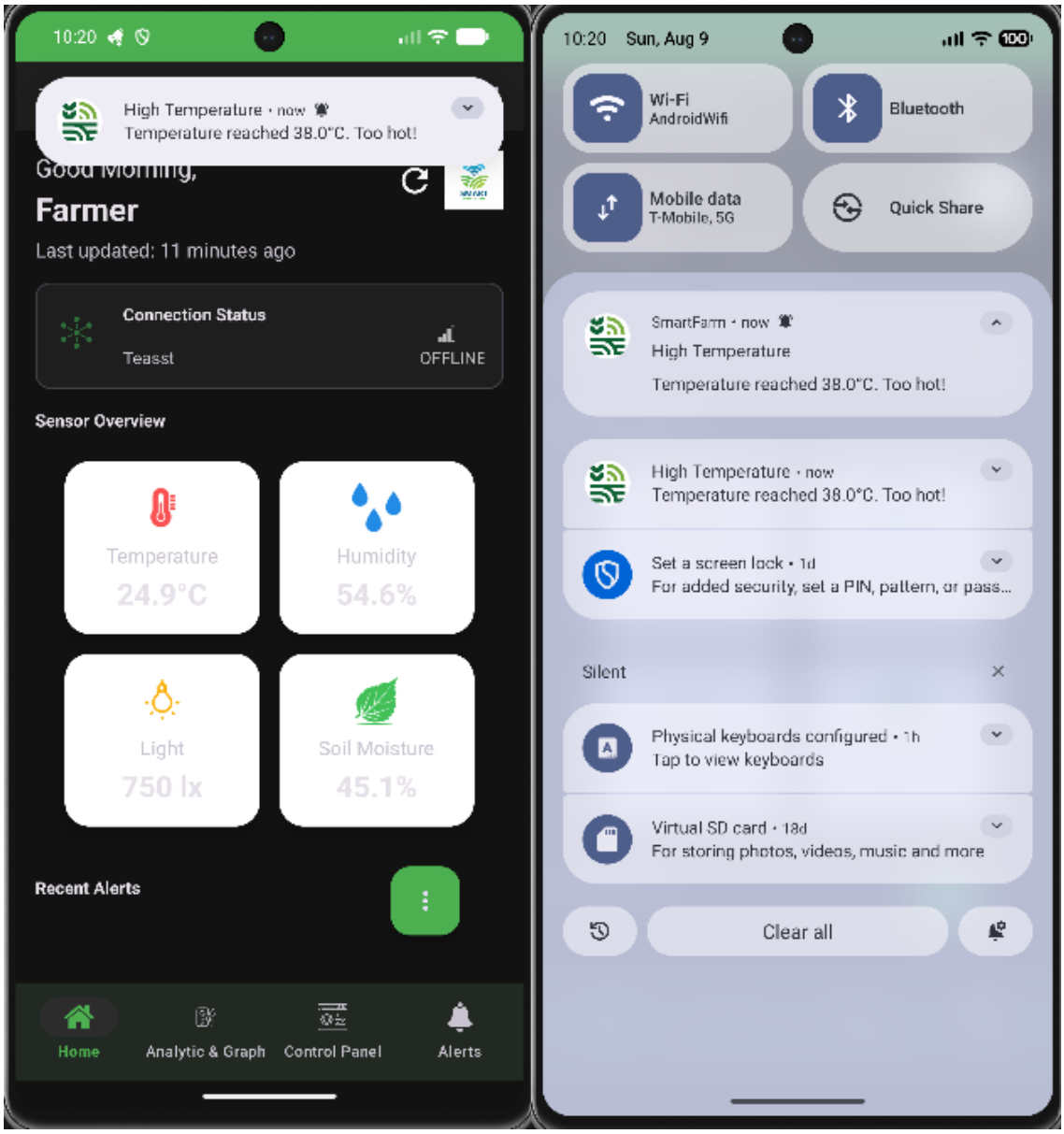
B. Snackbar


3:31
SmartFarm
Account
Alerts
Alert Summary
2 active alerts detected in your farm environment.
Notification Settings
Enable Alert Notifications
Critical Alerts Only
Active Alerts
High Temperature
Temperature reached 39°C. This exceeds the safe threshold for seedlings.
Detected: Today, 2:30 PM
Low Soil Moisture
Soil moisture dropped to 18%. Watering may be required.
Detected: Today, 2:30 PM
System Status
Other sensors are currently within the safe range.
Critical Alerts Only

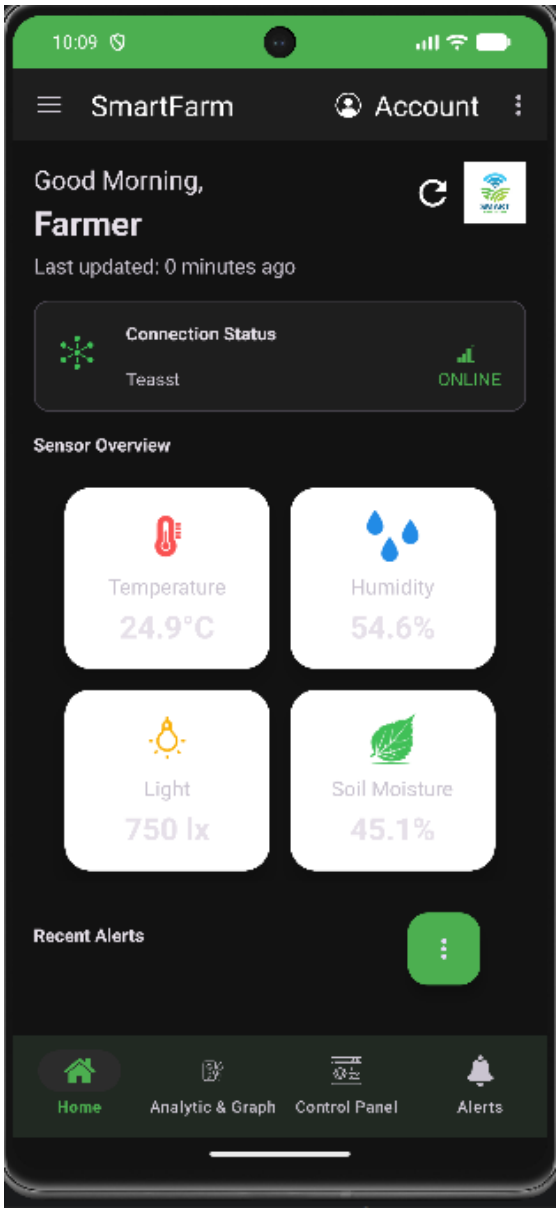
C. Toast


3:10
SmartFarm
Account
Help and Support
SmartFarm helps you monitor farm conditions and receive alerts when environmental conditions require attention.
Dashboard
View temperature, humidity, soil moisture, and light information in real time.
Alerts
An alert is displayed when a monitored environmental value exceeds its configured safe threshold.
Status Indicators
Green — Normal conditions
Yellow — Warning conditions
Red — Critical conditions
Frequently Asked Questions
Why did I receive an alert?
Alerts appear when sensor values exceed safe thresholds.
Contact Support
The Dashboard displays real-time farm sensor information.
Home
Alerts

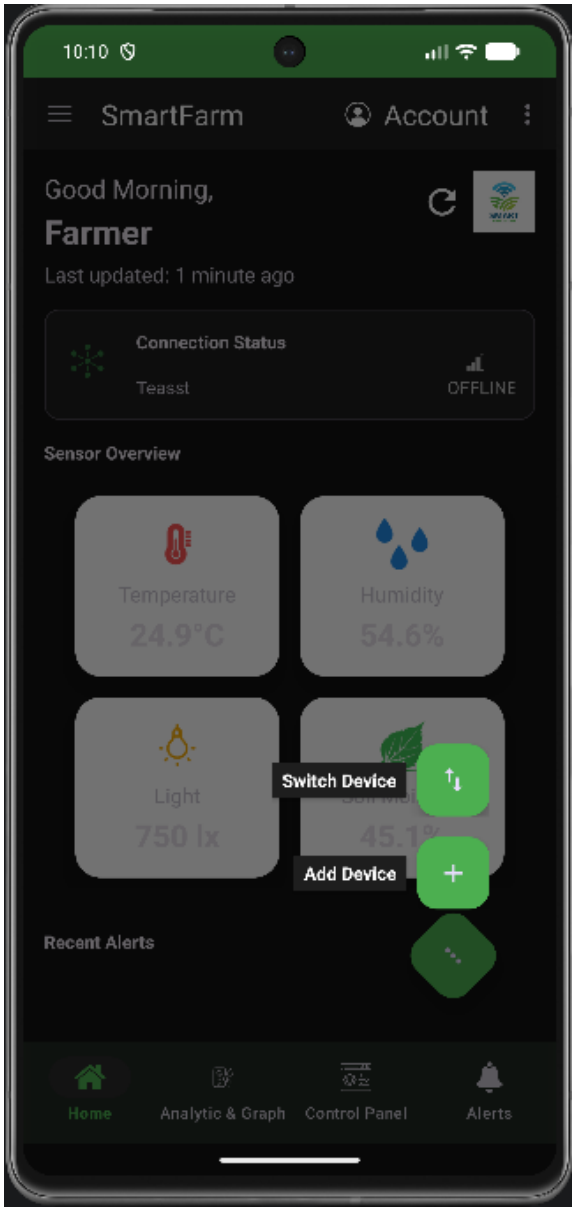
D. Notification



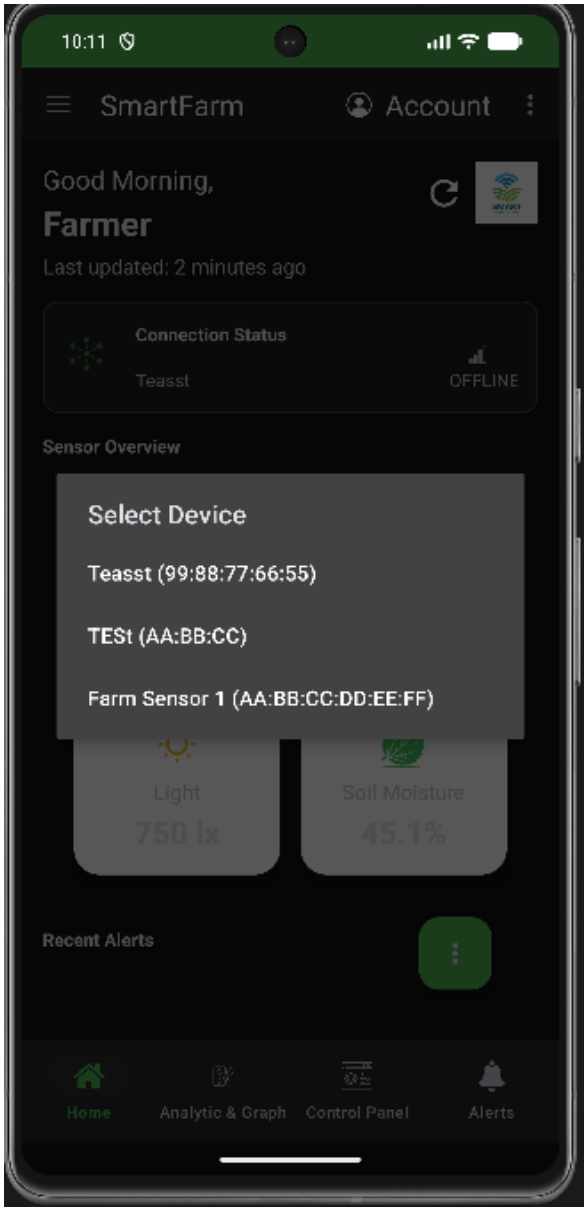
Screenshot showing the fab button and how the functionality fits into your app.



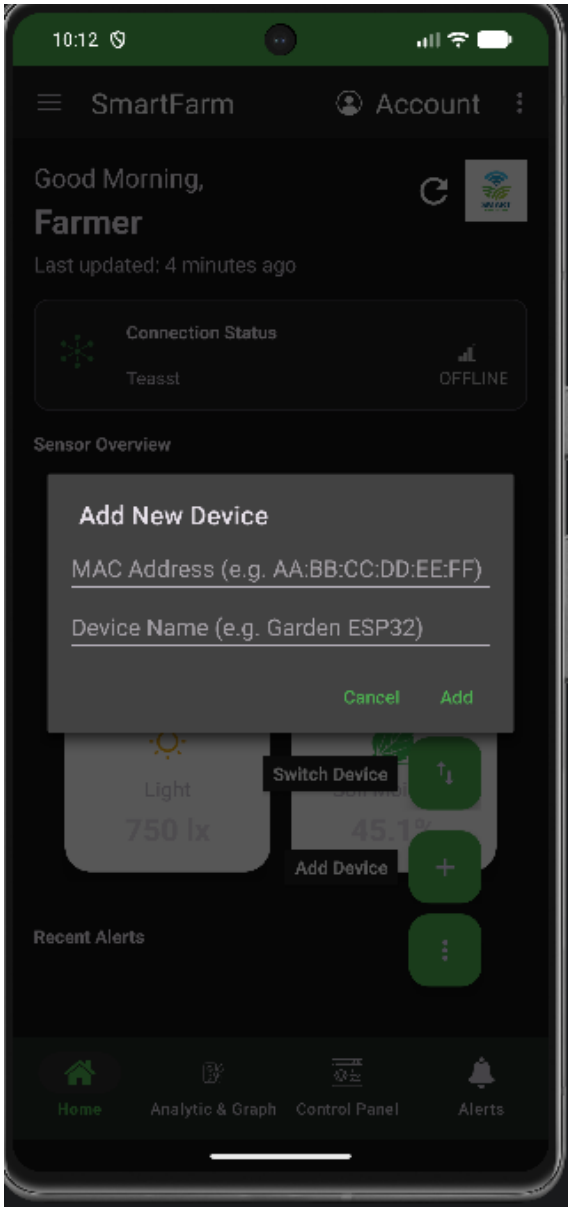
(Three dot floating action button)



(When user click the button, screen will dim, allow more focus on the functions)



(“Switch device” allow user to select device from “Add Device”)



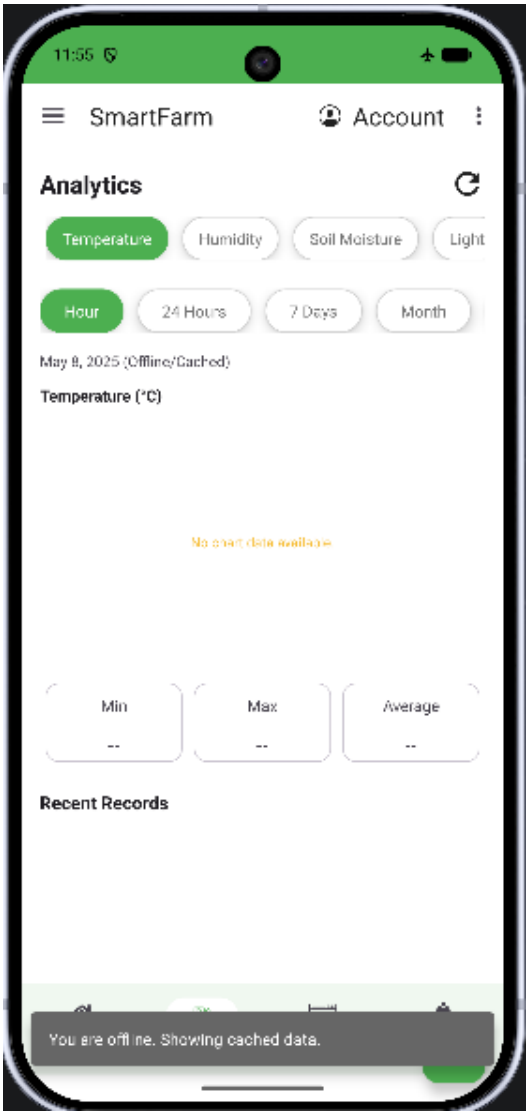
(“Add Device” allows users to add new devices. MAC address is required; device name can be anything.

The fab button help user add and manage their device. Each device has its own datas and the app will change accordingly. For example, the graph will be different based on what device users choose, and the Control panel will only affect the chosen device.

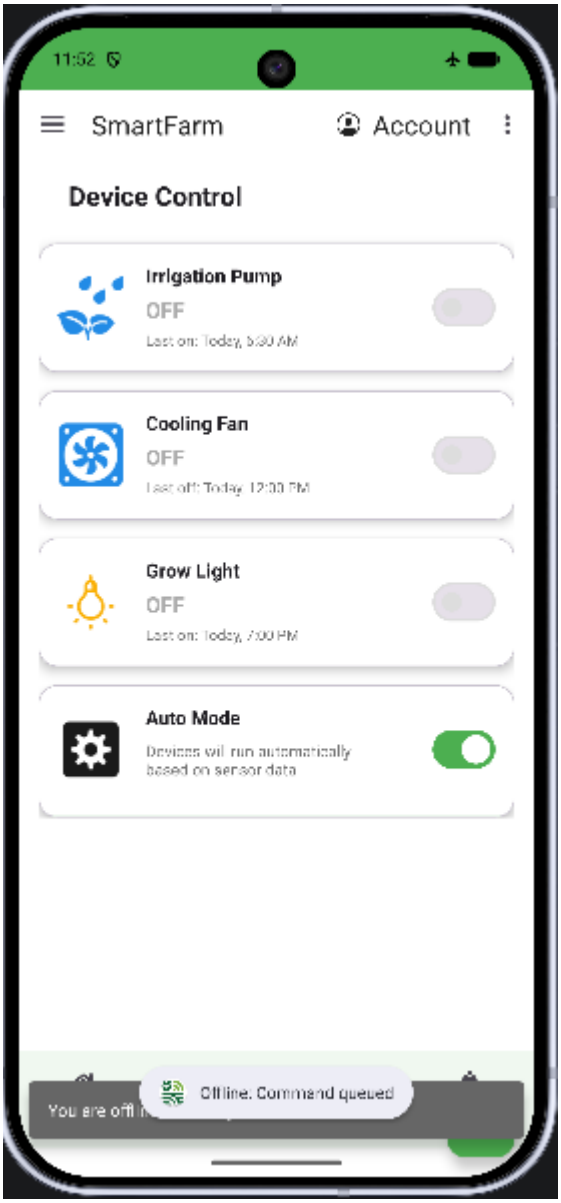
Document what feature will work off-line. Should be meaningful feature. Will be tested, when in Airplane mode. Provide screenshot

1. Data Visibility (Dashboard & Analytics)

- Cached Sensor Readings: The dashboard (Home screen) will continue to display the last received sensor data. When offline, the app adds a "(Cached)" label next to the status to let user know the data isn't live.
- Historical Data: User can still browse Analytic Graphs and historical records. The app retrieves these from a local database (Firestore cache), and labels them as "(Offline/Cached)".

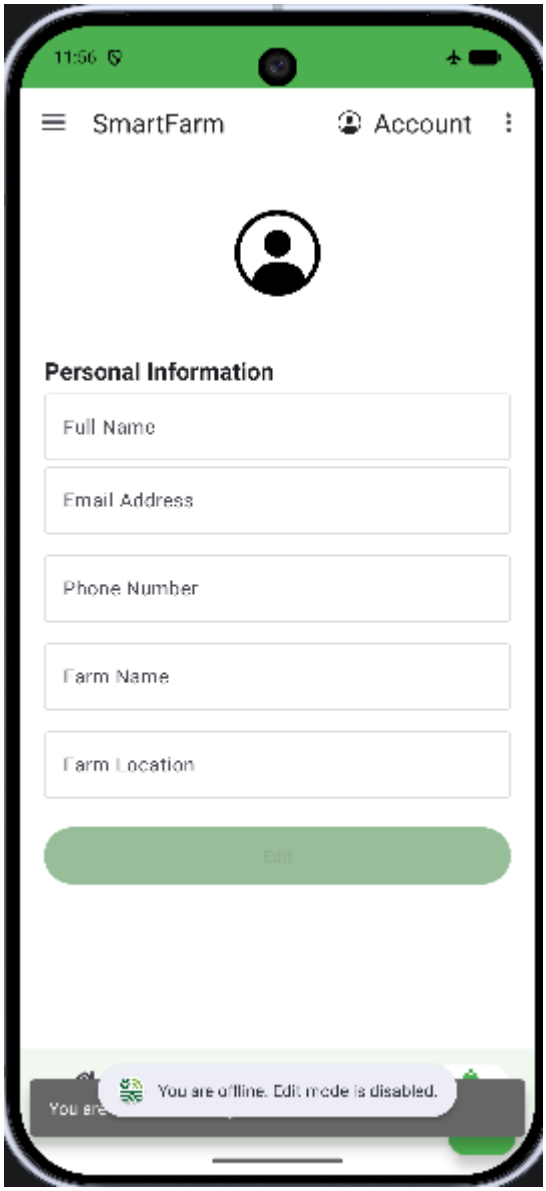


2. Device Control (Queued Commands)
    - Offline Switching: If user toggle the Irrigation Pump, Fan, or Lights while offline, the app will show a message: "Offline: Command queued".

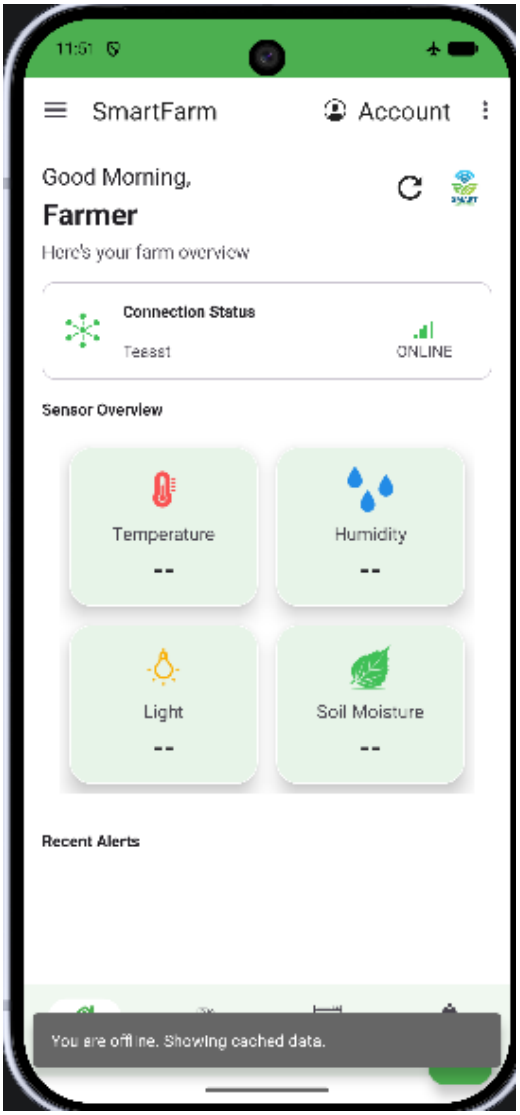


- Auto-Sync: User commands are saved locally and will be automatically sent to user farm's physical devices the moment user phone regains an internet connection.

3. Alerts & Notifications
   - Local Monitoring: The app continues to monitor the last cached sensor data. If the last data received before going offline indicates a problem (like high temperature), the system will still save that alert to user local history and can trigger a phone notification.
4. Account Screen
   - The profile system prioritizes a seamless user experience by caching data locally and gracefully handling offline scenarios while preventing data corruption by blocking edits during offline periods.



5. Continuity
    - Device Memory: The app uses persistent storage to remember which farm device user were last managing, so it will load correctly even if user start the app without internet.
    - Offline Banner: User will see a clear status banner at the bottom of the screen ("You are offline. Showing cached data.") so user always know the current state of connection.



Exception Handler

Approach for AccountFragment:

- **User-facing exceptions** are converted to Toast messages
- **Network exceptions** trigger offline mode
- **Validation exceptions** are handled through LiveData error states
- **UI exceptions** are prevented through state management

Exception Type	Location	Handling Method
Network not available	checkNetworkStatus()	Disables edit mode, shows Toast, sets offline flag
Null Context	checkNetworkStatus()	Returns early without crashing
Null User data	updateUI()	Sets empty strings instead of null
Image loading errors	updateUI()	Glide error placeholder (R.drawable.account)
Activity destroyed	setupImagePicker()	ActivityResultLauncher handles lifecycle
Validation errors	Various observers	Shows errors in TextInputLayout, disables save button

Post-Mortem, Project Review Meeting, document the below:

A. Performance review of the project

Cost

The project was completed within the available budget, with no major additional costs. The main expenses were development tools, sensors, and the account required to upload the app. Overall, the project remained within budget.

Schedule

The project experienced some delays due to miscommunication regarding the app's UX/UI, and some functions were not approved by the team. The team spent approximately three weeks discussing and developing solutions to address these issues. Despite the challenges, the team prioritized the core features and completed the main functionality by the deadline. Overall, there were some delays during the project, but the key requirements were successfully completed on time.

Quality

The core Smart Farm features, including user authentication, sensor monitoring, alerts, and help functionality, were successfully implemented. Some compromises were made

in automated UI testing and the chatbot feature due to time constraints. Overall, the project achieved **good quality in its core functionality**, with opportunities for improvement in accessibility, error handling, and overall user-friendliness.

B. Time Management Evaluate

Member	Time Management	Assessment
Chau Vi Cuong Nguyen	Excellent	Consistent commits Work distributed evenly No last-minute rushes
Hoang Tran	Good	UI work completed on time; No last-minute rushes; Consistent progress
Tuan Anh Pham	Good	Function and UI work finished on time; No last-minute rushes; Helped with additional tasks
Truong An Nguyen	OK	Diagrams and retro completed on time; Occasionally had last-minute rushes

C. Issues with the quality or compromises along the way

We faced some quality issues during development, mainly with Firebase integration, sensor communication, dependencies, and testing. Due to time constraints, some features were kept simple, such as using Toasts and Snackbars for notifications instead of implementing more advanced UI feedback.

We also had limited automated testing for Android UI components and relied on some manual testing. However, we prioritized the core functionality of the Smart Farm system and resolved major technical issues before the final implementation.

D. Lessons learned, mistakes, and areas of improvement.

Lessons Learned

The project helped the team gain practical experience with Android development, Firebase, IoT devices, sensors, RFID, and communication between hardware and software.

Another important lesson was the importance of clear communication between team members. Dividing the project into smaller tasks made it easier for members to work on different components simultaneously.

Mistakes

The team made several mistakes during development, including not consistently using feature branches, which caused merging conflicts.

We also experienced Firebase initialization issues and Robolectric version mismatches during testing. In addition, some tests were accidentally run as Android instrumented tests instead of JUnit unit tests.

We addressed these issues by improving file coordination, updating dependency configurations, initializing Firebase correctly, using JUnit-only tests for application logic, and moving tests to the correct `src/test` directory.

Areas for Improvement

Future versions of the project could improve the following areas:

- Increase automated unit and UI test coverage.
- Improve error handling for Firebase and sensor communication.
- Replace some Toast messages with more informative dialogs or Snackbars where appropriate.
- Improve the visual consistency of the application.
- Perform integration testing earlier in the development process.
- Separate application logic from UI code to make testing easier.

E. Metting attendances

Member	Attended	Notes
Chau Vi Cuong Nguyen	Yes	Presented work completed Assigned work for each member
Hoang Tran	Yes	Presented UI/UX updates Help with additional tasks
Tuan Anh Pham	Yes	Presented work completed Advised on how UI/UX could improve
Truong An Nguyen	Yes	Presented work completed Work on diagrams, notes and tests

F. Technical Debt solutions

Technical Debt Item	Description	Solutions
Hardcoded String	UI text was hardcoded in XML	Moved all strings to strings.xml, provide French translation
Validation Logic in UI	LoginActivity handled validation	Extracted to LoginValidator.java (SRP)

Large Activity Classes	Activities with 200+ lines	Split into ViewModel + Fragment/Activity (MVVM)
No Unit Tests	No Unit Tests	Added 3 JUnit classes, espresso test 5+ cases, and Robletetric

Coding standards and their usage

The refactoring of AlertView.java and creation of NotificationPermissionHelper.java applied several industry-standard coding practices to improve code quality, maintainability, and user experience.

Standard 1: Single Responsibility Principle (SRP)

Every class should have one, and only one, reason to change. A class should handle a single responsibility.

Before Refactoring:

```
1. // AlertView.java - This class does TOO MUCH
2. public class AlertView extends Fragment {
3.     // UI responsibilities
4.     private Switch switchNotifications;
5.     private Switch switchCriticalOnly;
6.
7.     // Permission state management
8.     private static final int NOTIFICATION_PERMISSION_REQUEST_CODE = 100;
9.     private boolean isProcessingPermission = false;
10.
11.    // Permission checking (should be separate)
12.    private boolean hasNotificationPermission() {
13.        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
14.            return ContextCompat.checkSelfPermission(
15.                    requireContext(),
16.                    Manifest.permission.POST_NOTIFICATIONS
17.            ) == PackageManager.PERMISSION_GRANTED;
18.        }
19.        return true;
20.    }
21.
22.    // Permission request logic (should be separate)
23.    private void requestNotificationPermission() {
24.        // 30+ lines of permission request logic
25.        if (ActivityCompat.shouldShowRequestPermissionRationale(...)) {
26.            new MaterialAlertDialogBuilder(requireContext())
27.                .setPositiveButton(...)
28.                .show();
29.        } else {
30.            requestPermissions(...);
31.        }
32.    }
33.
34.    // Permission result handling (should be separate)
35.    @Override
36.    public void onRequestPermissionsResult(...) {
37.        // Handle permission results
38.        if (requestCode == NOTIFICATION_PERMISSION_REQUEST_CODE) {
39.            // Update UI based on permission
40.        }
```

```
    }

    // UI state management
    private void showSnackbar(int messageResId, boolean isGranted) { ... }
    private void showToast(int messageResId) { ... }
}

```

After Refactoring: SRP Applied

```
// AlertView.java - NOW ONLY HANDLES UI
public class AlertView extends Fragment {
    // ONLY UI-related fields
    private Switch switchNotifications;
    private Switch switchCriticalOnly;
    private NotificationPermissionHelper permissionHelper;

    @Override
    public View onCreateView(...) {
        View view = inflater.inflate(R.layout.alert, container, false);

        // UI initialization only
        this.switchNotifications = view.findViewById(R.id.switch_notifications);
        this.switchCriticalOnly = view.findViewById(R.id.switch_critical_only);

        // Permission handling delegated to helper
        permissionHelper = new NotificationPermissionHelper(this,
            new NotificationPermissionHelper.PermissionCallback() {
                @Override
                public void onPermissionGranted() {
                    // ONLY UI updates here
                    switchNotifications.setChecked(true);
                    switchCriticalOnly.setEnabled(true);
                }

                @Override
                public void onPermissionDenied() {
                    // ONLY UI updates here
                    switchNotifications.setChecked(false);
                    switchCriticalOnly.setEnabled(false);
                }
            }
        );

        // Restore UI state
        switchNotifications.setChecked(permissionHelper.getNotificationEnabled());
        switchCriticalOnly.setEnabled(switchNotifications.isChecked());
        switchCriticalOnly.setChecked(permissionHelper.getCriticalOnlyEnabled());

        // Clean UI listener
        switchNotifications.setOnCheckedChangeListener((buttonView, isChecked) -> {
            if (isChecked) {
                permissionHelper.handleEnableNotification(); // Delegate to helper
            } else {
                permissionHelper.setNotificationEnabled(false);
                switchCriticalOnly.setEnabled(false);
            }
        });

        return view;
    }
}

```

```
// NotificationPermissionHelper.java - DEDICATED TO PERMISSIONS
public class NotificationPermissionHelper {
    // ONLY permission-related fields and logic
```

```
4.
5.      private final Context context;
6.      private final Fragment fragment;
7.      private final PermissionCallback callback;
8.      private ActivityResultLauncher<String> notificationPermissionLauncher;
9.      private SharedPreferences prefs;
10.
11.     // Single responsibility: Check permission
12.     public boolean hasNotificationPermission() {
13.         if (requiresNotificationPermission()) {
14.             return ContextCompat.checkSelfPermission(
15.                     context,
16.                     Manifest.permission.POST_NOTIFICATIONS
17.             ) == PackageManager.PERMISSION_GRANTED;
18.         }
19.         return true;
20.     }
21.
22.     // Single responsibility: Request permission
23.     public void requestNotificationPermission() {
24.         if (hasNotificationPermission()) {
25.             return;
26.         }
27.
28.         boolean requestedBefore = prefs.getBoolean(NOTIFICATION_PERMISSION_REQUESTED,
false);
29.         if (!requestedBefore) {
30.             prefs.edit().putBoolean(NOTIFICATION_PERMISSION_REQUESTED, true).apply();
31.             notificationPermissionLauncher.launch(Manifest.permission.POST_NOTIFICATIONS);
32.         } else if (fragment.shouldShowRequestPermissionRationale(...)) {
33.             notificationPermissionLauncher.launch(Manifest.permission.POST_NOTIFICATIONS);
34.         } else {
35.             Toast.makeText(context, R.string.manual_permission, Toast.LENGTH_LONG).show();
36.             openAppSettings();
37.         }
38.     }
39.
40.     // Single responsibility: Handle enable flow
41.     public void handleEnableNotification() {
42.         if (requiresNotificationPermission()) {
43.             if (hasNotificationPermission()) {
44.                 setNotificationEnabled(true);
45.                 callback.onPermissionGranted();
46.             } else {
47.                 requestNotificationPermission();
48.             }
49.         } else {
50.             setNotificationEnabled(true);
51.             callback.onPermissionGranted();
52.         }
53.     }
54. }
55.
```

Standard 2: Using Modern Android APIs

Before: Deprecated API Usage

```
1. // AlertView.java - Using deprecated onRequestPermissionsResult
2.
3. @Override
4. public void onRequestPermissionsResult(
5.         int requestCode,
6.         @NonNull String[] permissions,
7.         @NonNull int[] grantResults) {
8.     // This method is deprecated in Android 13+
```

```
    super.onRequestPermissionsResult(requestCode, permissions, grantResults);

    if (requestCode == NOTIFICATION_PERMISSION_REQUEST_CODE) {
        if (grantResults.length > 0 && grantResults[0] == PackageManager.PERMISSION_GRANTED)
{
            switchNotifications.setChecked(true);
            switchCriticalOnly.setEnabled(true);
        } else {
            switchNotifications.setChecked(false);
            switchCriticalOnly.setEnabled(false);
        }
    }
}

// Using ActivityCompat - more verbose
if (ActivityCompat.shouldShowRequestPermissionRationale(
        requireActivity(),
        Manifest.permission.POST_NOTIFICATIONS
)) {
    // ... show dialog
}

```

After: Modern ActivityResultLauncher API

```
// NotificationPermissionHelper.java - Using modern API

public class NotificationPermissionHelper {
    // Modern ActivityResultLauncher
    private ActivityResultLauncher<String> notificationPermissionLauncher;

    public void initializePermissionLauncher() {
        // Clean, modern API
        notificationPermissionLauncher = fragment.registerForActivityResult(
                new ActivityResultContracts.RequestPermission(),
                isGranted -> {
                    if (isGranted) {
                        setNotificationEnabled(true);
                        callback.onPermissionGranted();
                        Toast.makeText(context, R.string.enable_notif,
Toast.LENGTH_SHORT).show();
                    } else {
                        setNotificationEnabled(false);
                        callback.onPermissionDenied();
                        Toast.makeText(context, R.string.deny_notif_perm,
Toast.LENGTH_SHORT).show();
                    }
                }
        );
    }

    public void requestNotificationPermission() {
        if (hasNotificationPermission()) {
            return;
        }

        boolean requestedBefore = prefs.getBoolean(NOTIFICATION_PERMISSION_REQUESTED,
false);
        if (!requestedBefore) {
            prefs.edit().putBoolean(NOTIFICATION_PERMISSION_REQUESTED, true).apply();
            // Clean launch - no request codes needed!
            notificationPermissionLauncher.launch(Manifest.permission.POST_NOTIFICATIONS);
        } else if
(fragment.shouldShowRequestPermissionRationale(Manifest.permission.POST_NOTIFICATIONS)) {
            notificationPermissionLauncher.launch(Manifest.permission.POST_NOTIFICATIONS);
        } else {
            Toast.makeText(context, R.string.manual_permission, Toast.LENGTH_LONG).show();
```

```
            openAppSettings();
        }
    }
}

```

Describe how DevOps would have helped your project.

What AI tool used and how it helped in your work.

Potential security vulnerability and future approach

```
// UserRepository.java
private final SharedPreferences sharedPreferences;
// ...
private void saveToCache(UserModel user) {
    sharedPreferences.edit()
            .putString(KEY_USER_ID, userId)
            .putString(KEY_USER_DATA, gson.toJson(user))
            .apply();
}

private UserModel loadFromCache() {
    String json = sharedPreferences.getString(KEY_USER_DATA, null);
    Type type = new TypeToken<UserModel>() {}.getType();
    return gson.fromJson(json, type);
}

```

Vulnerability: Sensitive user data (full name, email, phone, farm details, profile image URL) is stored in plain text in SharedPreferences. This data is accessible to any app with root access, or through backup/restore operations. The Gson serialization also exposes all user data fields.

Future Fix Approach:

1. **Replace SharedPreferences with EncryptedSharedPreferences**
    - Use Android Security Crypto library
    - Encrypt both keys and values using AES-256-GCM
    - Implement secure key generation using Android Keystore
    - Store encryption keys in hardware-backed keystore when available
2. **Implement Data Classification**
    - Separate sensitive data (PII) from non-sensitive data
    - Apply different encryption levels based on sensitivity

- Encrypt entire UserModel object before storage

3. **Add File-Level Encryption**
    - Use EncryptedFile from Android Security Crypto for larger data
    - Apply additional obfuscation layer for ultra-sensitive fields
4. **Implement Secure Key Rotation**
    - Rotate encryption keys periodically
    - Re-encrypt data during key rotation
    - Maintain key versioning for backward compatibility
5. **Add Tamper Detection**
    - Store data with HMAC signature
    - Verify integrity on each load
    - Clear cache if tampering detected

Screen Tracking Analytics

My solution implements a **developer-only statistics viewing system** that leverages the ScreenTrackingManager to display screen usage analytics. The approach focuses on minimal UI impact while providing comprehensive data access for developers.

Core Implementation Approach

1. Data Collection Strategy

The ScreenTrackingManager is implemented with a **document-per-screen** structure in FireStore:

```
/**
 * Track a screen click - Stores daily and monthly aggregates
 * Data structure:
 * screen_tracking/
 *    └── {screen_name}/
 *        ├── total: 150
 *        ├── daily/
 *        │   ├── 2026-02-01: 45
 *        │   ├── 2026-02-02: 38
 *        │   └── ...
 *        ├── monthly/
 *        │   ├── 2026-01: 120
 *        │   ├── 2026-02: 95
 *        │   └── ...
 *        └── last_updated: timestamp
 */
```

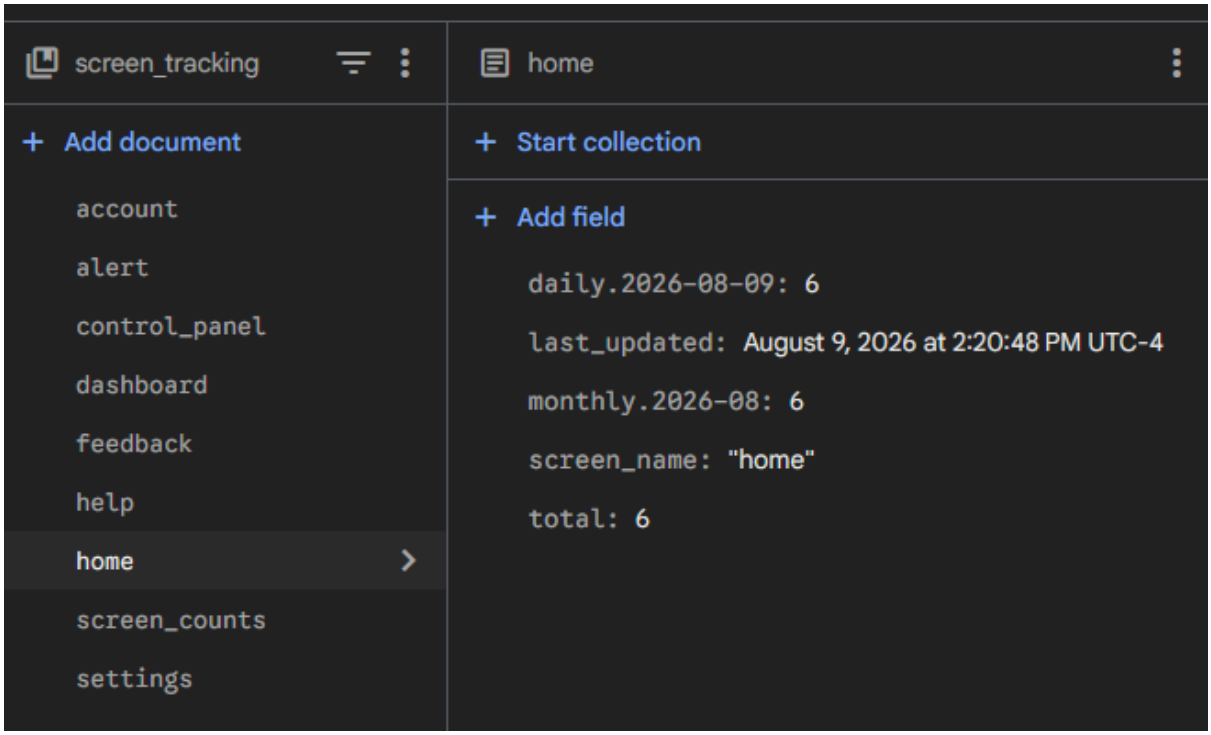
Key Design Decisions:

- Each screen has its own document
- Daily and monthly aggregates are stored as nested maps
- total field maintains cumulative count
- last_updated timestamp for tracking freshnes

The solution integrates seamlessly into the existing MainActivity.java without affecting regular users:

```
1. // Tracking already implemented throughout MainActivity
2. trackingManager.trackScreenClick(ScreenTrackingManager.SCREEN_HOME);
3. trackingManager.trackScreenClick(ScreenTrackingManager.SCREEN_DASHBOARD);
4. trackingManager.trackScreenClick(ScreenTrackingManager.SCREEN_SETTINGS);
5. trackingManager.trackScreenClick(ScreenTrackingManager.SCREEN_ACCOUNT);
6. trackingManager.trackScreenClick(ScreenTrackingManager.SCREEN_ALERT);
7. trackingManager.trackScreenClick(ScreenTrackingManager.SCREEN_FEEDBACK);
8. trackingManager.trackScreenClick(ScreenTrackingManager.SCREEN_CONTROL_PANEL);
```

Finally, developers can access FireStore collections to view statistics



Test Cases Creation

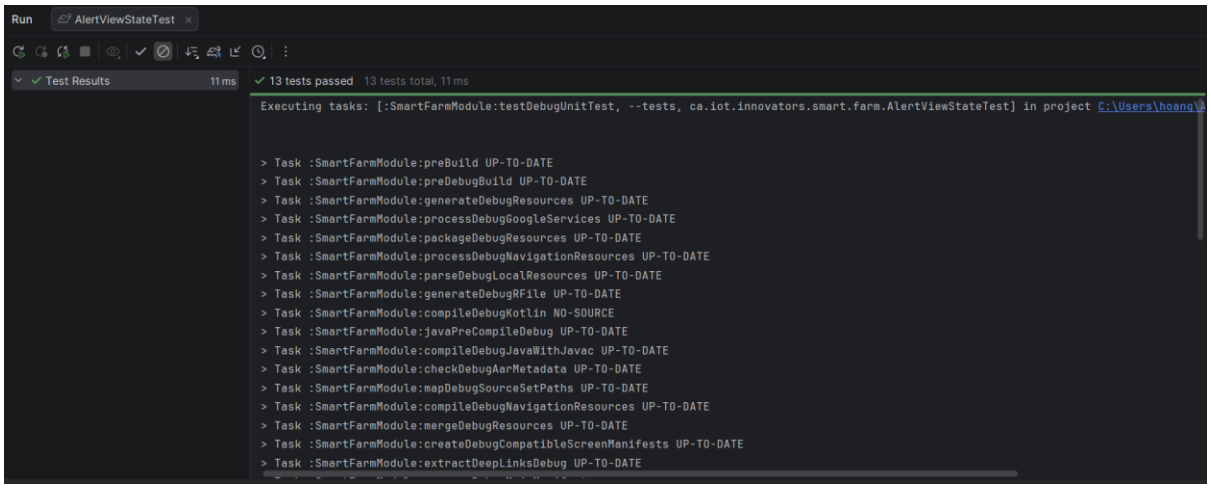
Describe your strategy for writing the test cases.

Strategy Component	Implementation
Unit Testing	InputCheckerTest, AlertViewStateTest
UI Testing	AccountFragmentEspressoTest, LoginUITest

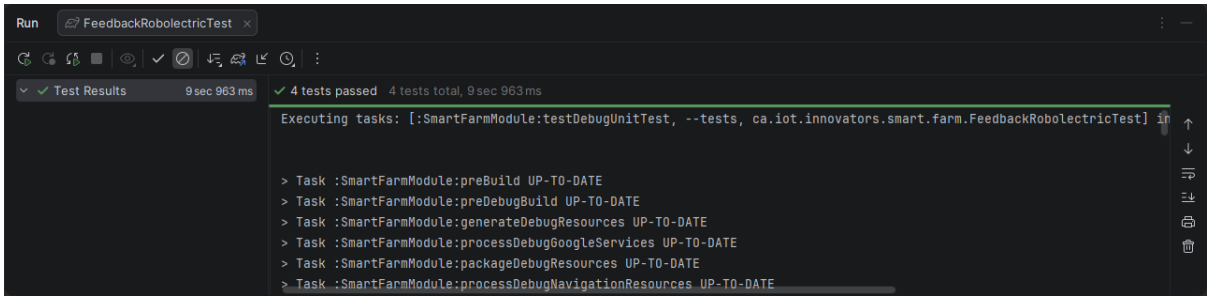
Strategy Component	Implementation
Behavior-Driven Testing	Given-When-Then pattern
Black-Box Testing	Testing inputs/outputs without internal knowledge
White-Box Testing	Testing state transitions and internal logic

Screenshots showing 3 classes junit test cases are passing.

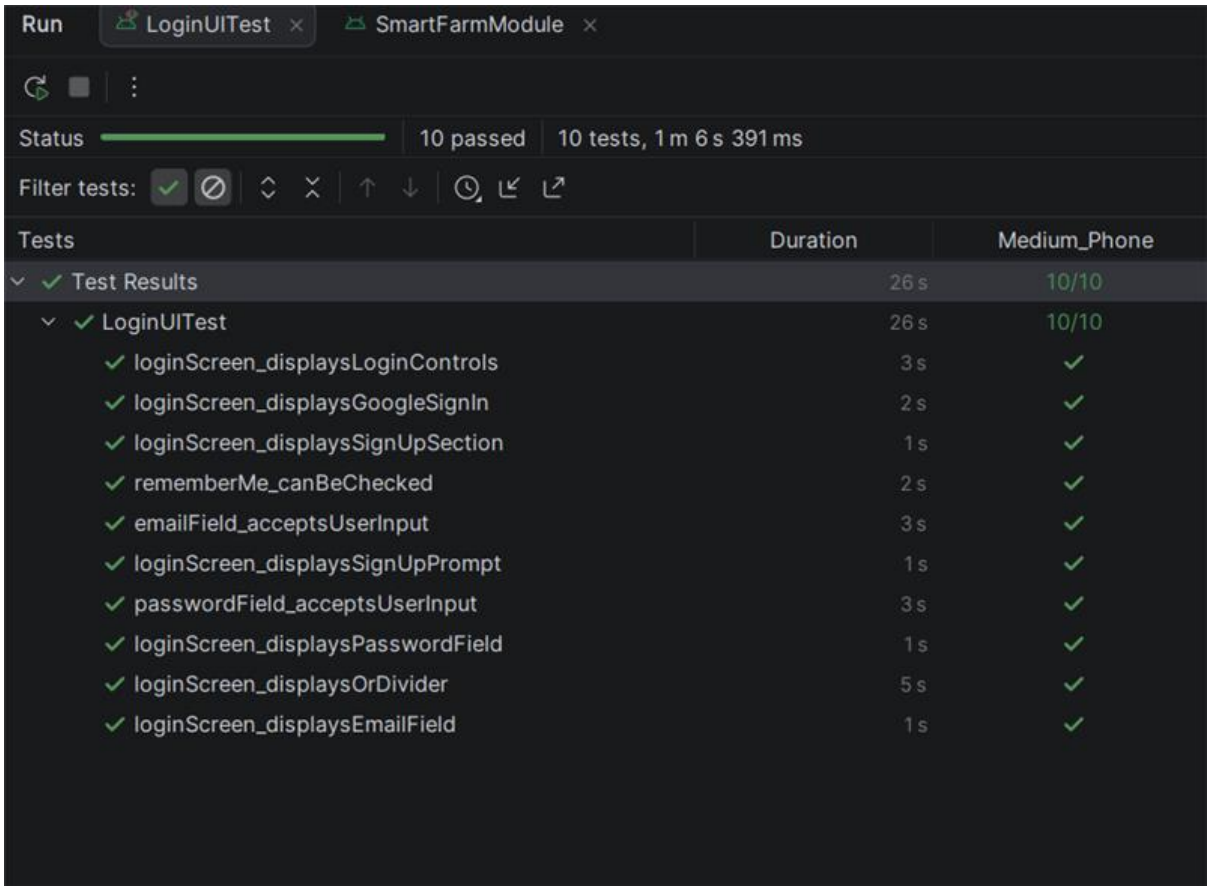
1. Alert class test:



Screenshot showing Robleteric test cases are passing



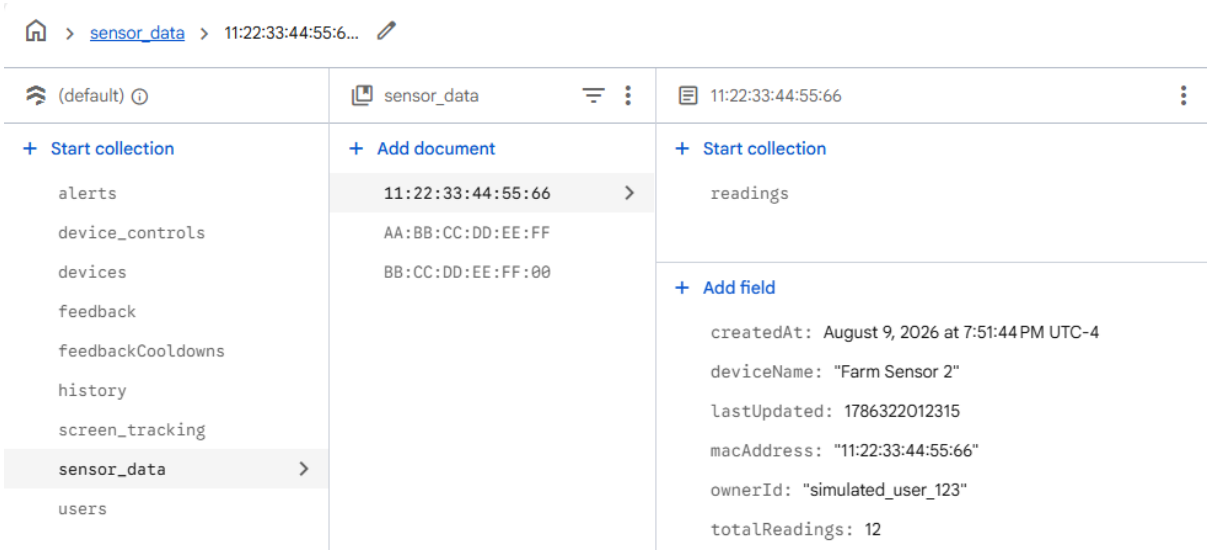
Screenshot showing Espresso test cases are passing.

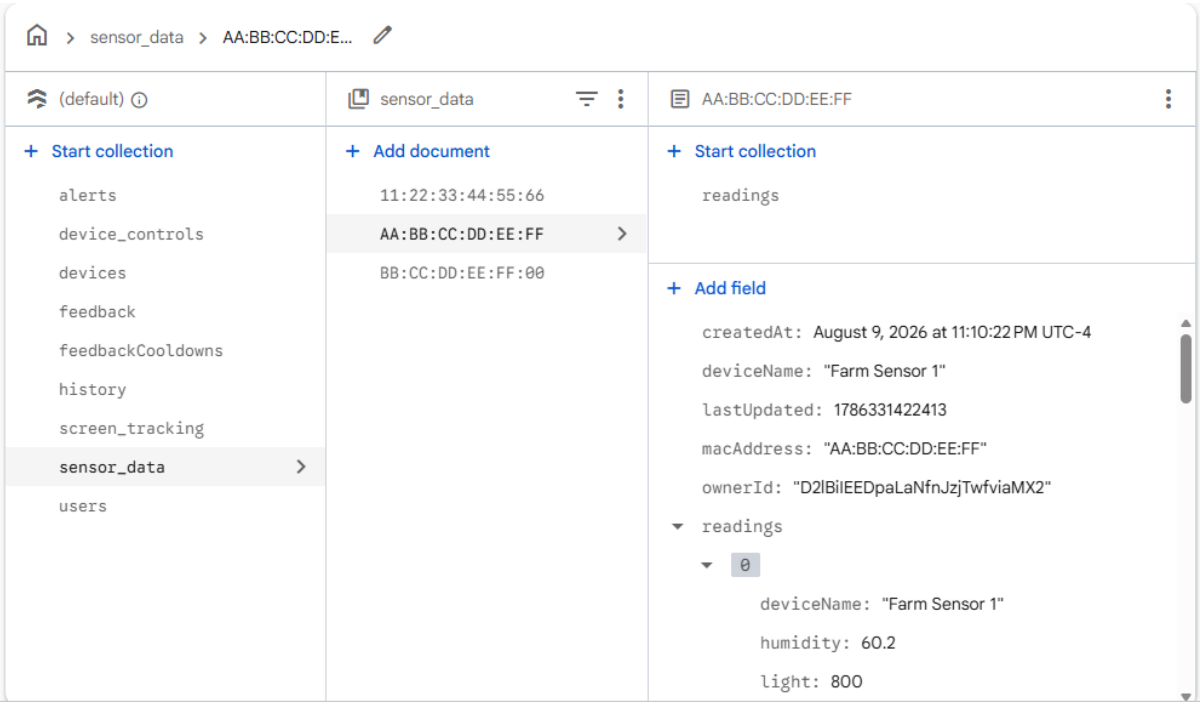


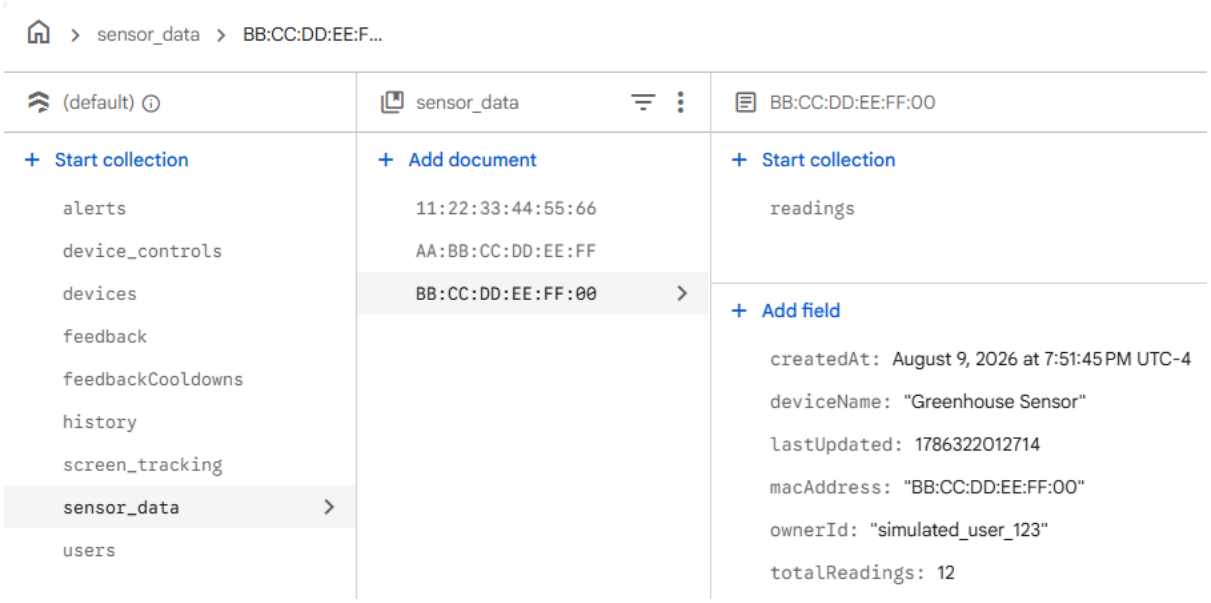
Screenshots showing the data in your DB, sensors data (at least 3 different sensors), and login data (including Google login) screenshot showing the data stored into the

DB from Feedback. Must have at least 3 different entries

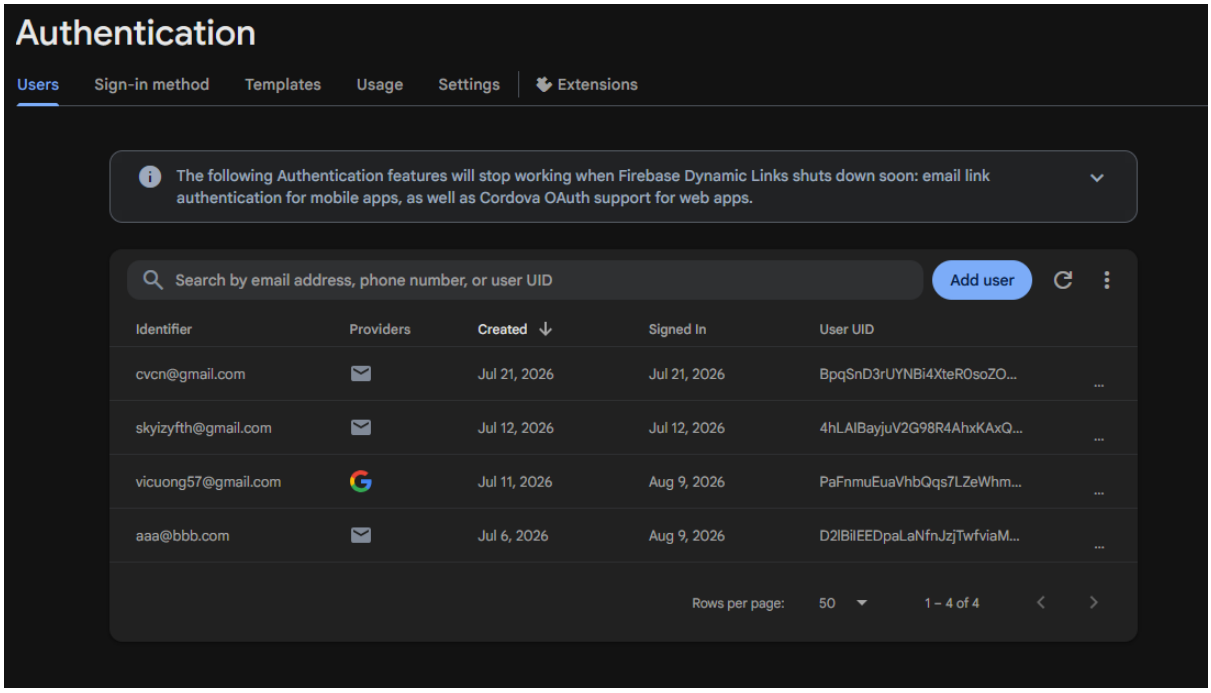
Screenshots showing sensors data:

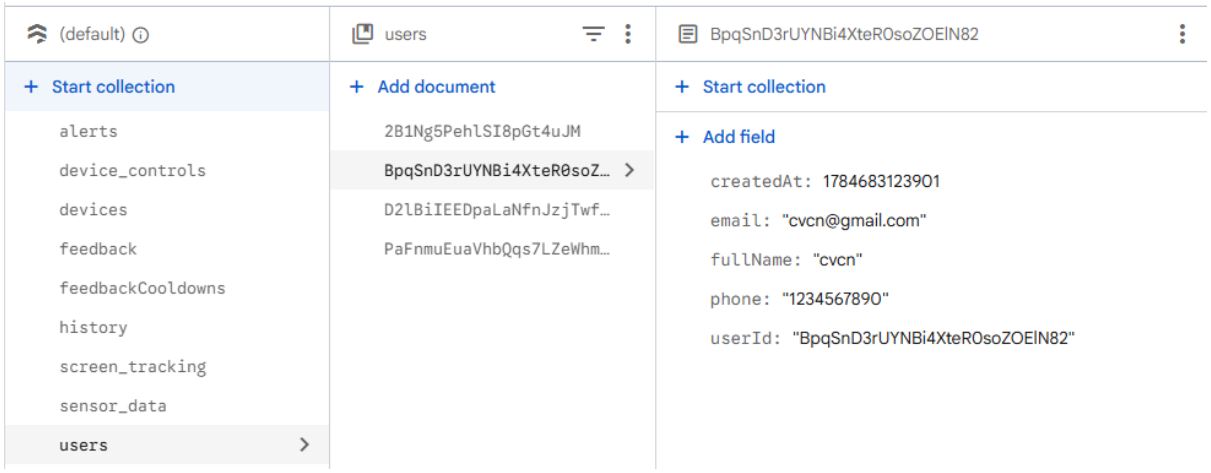


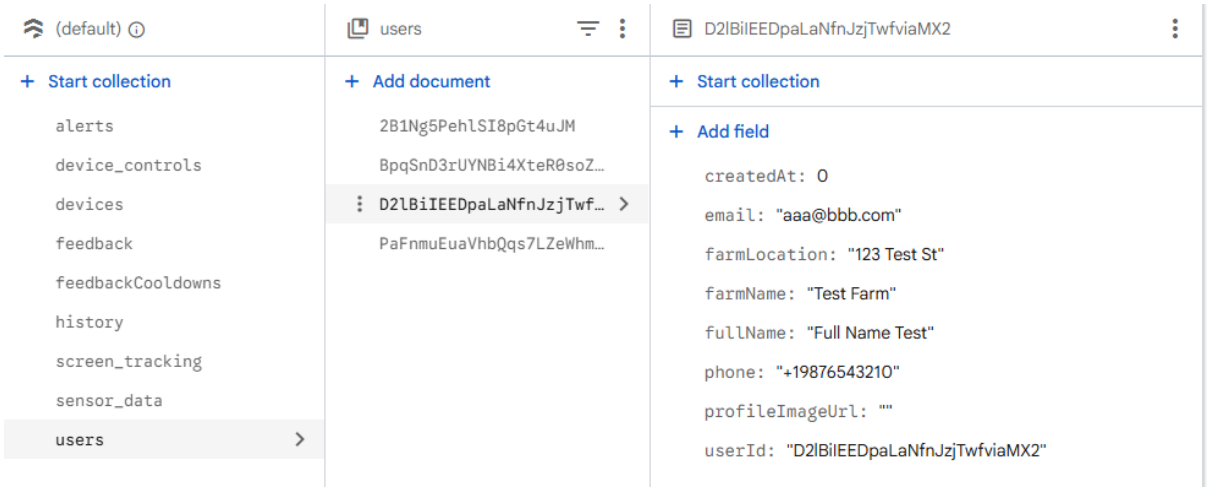


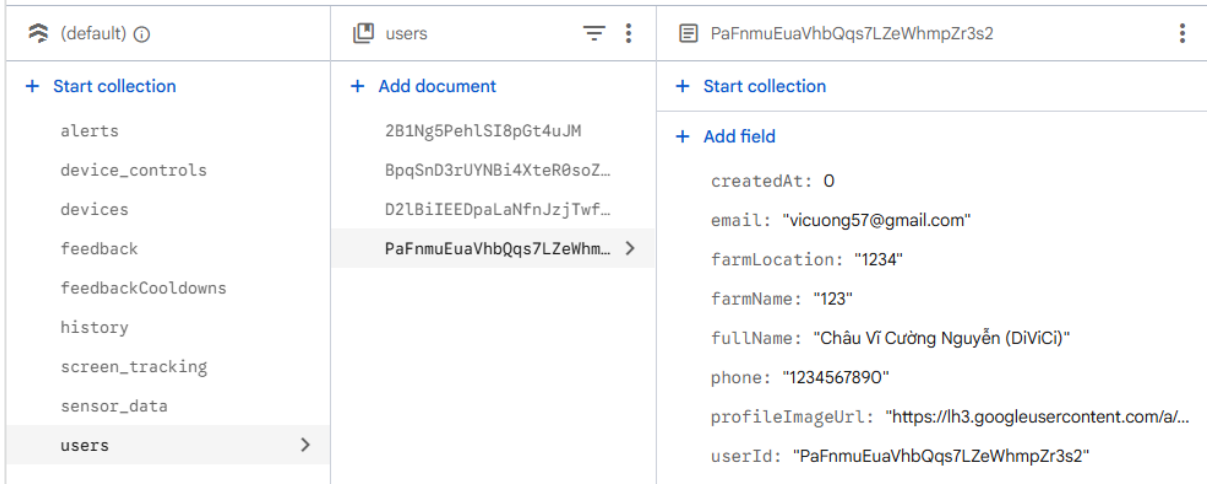


Screenshots showing login data:



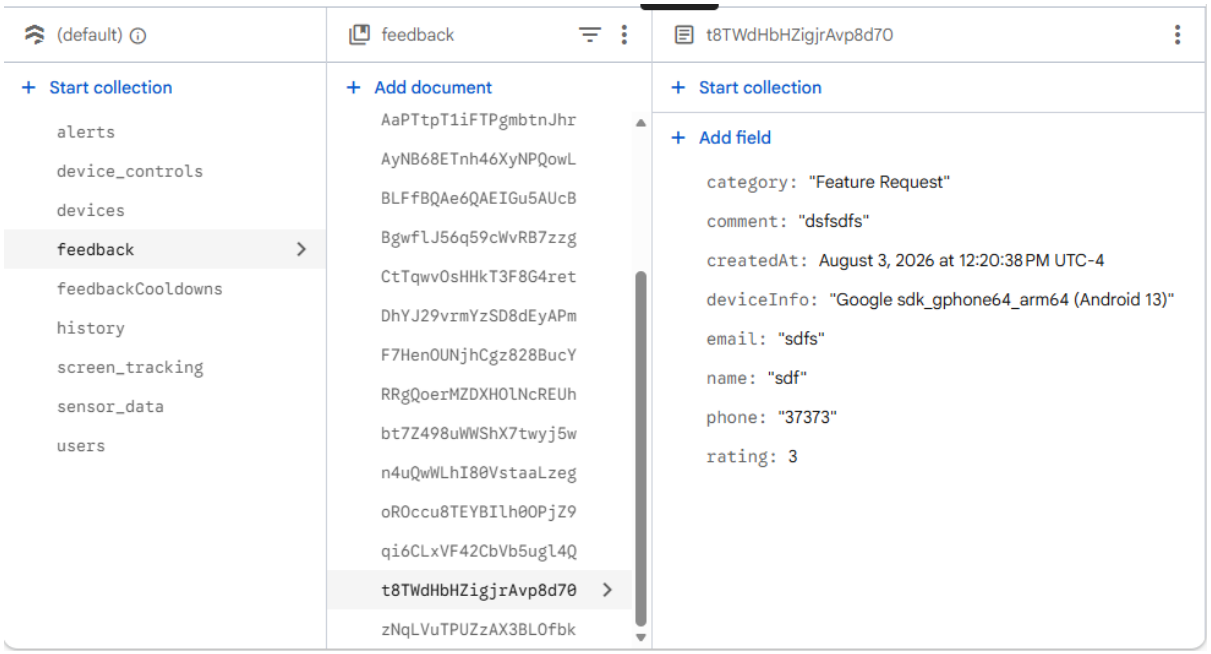


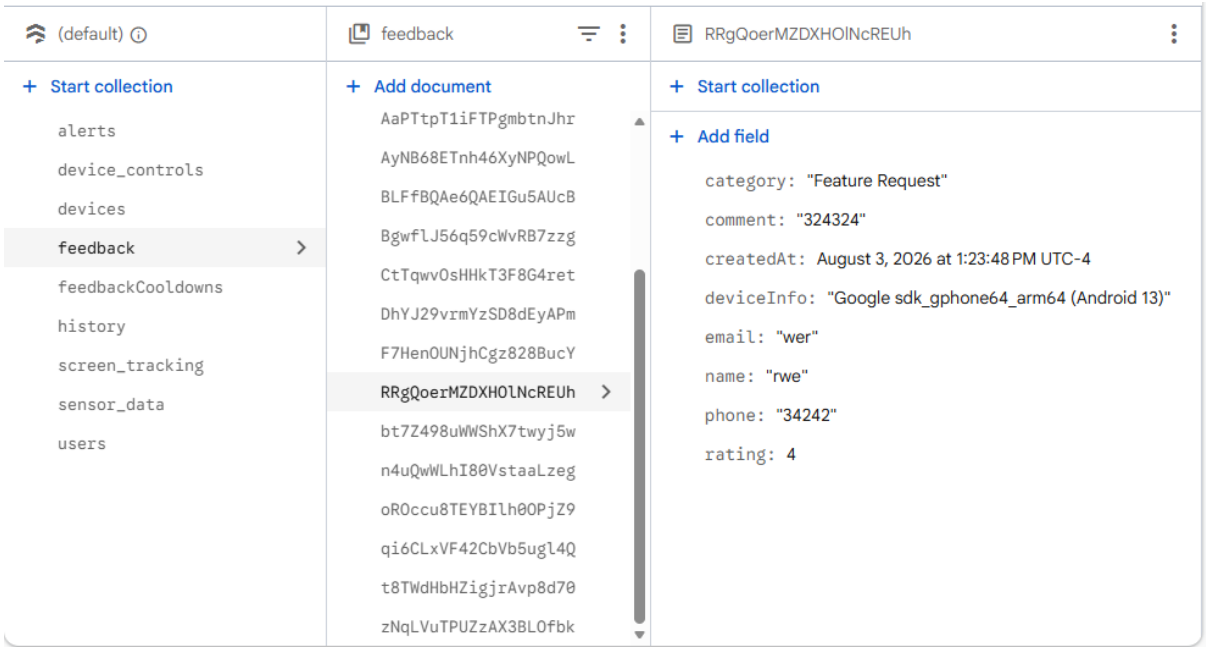


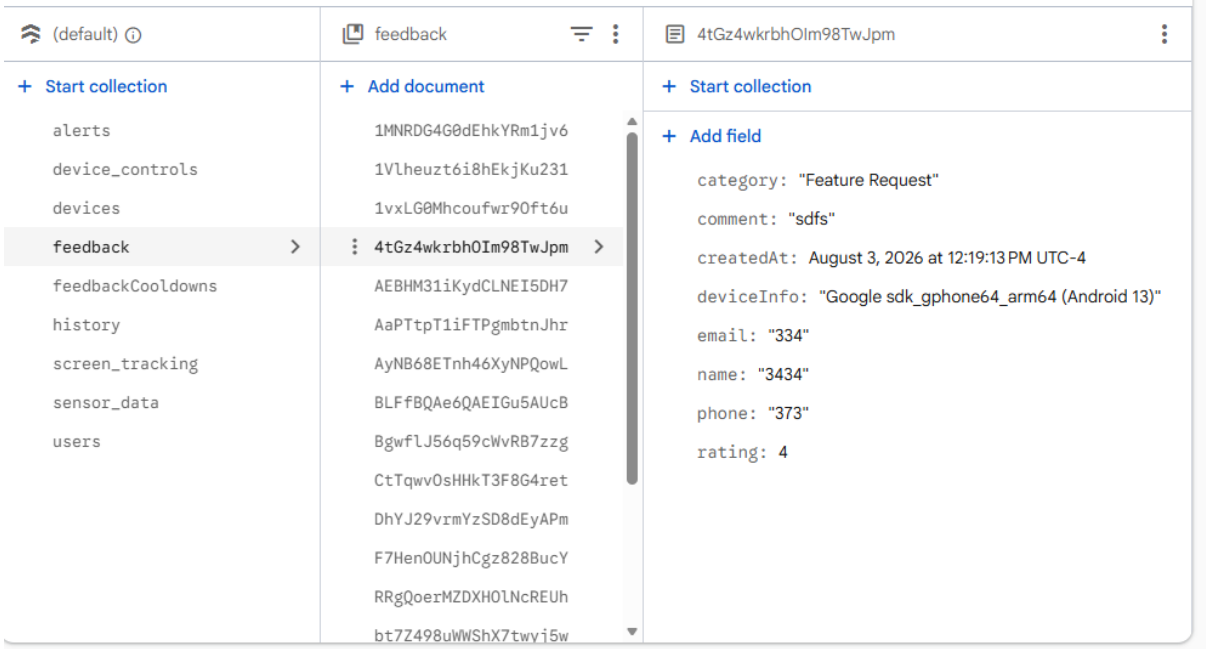


(this one is google login)

Screenshots showing data stored into the DB from Feedback:







Simulation Script Documentation

Overview

Single-execution Cloud Run Job that generates and uploads **one** sensor reading per run. Designed to be triggered by Cloud Scheduler (for now run every 10 minutes).

Data Generation

Normal Values

- **Temperature**: 20-30°C (random walk ±0.5)
- **Humidity**: 40-70% (random walk ±0.5)

- **Light**: 300-1000 lux (random walk ±0.5)
- **Soil Moisture**: 35-65% (random walk ±0.5)

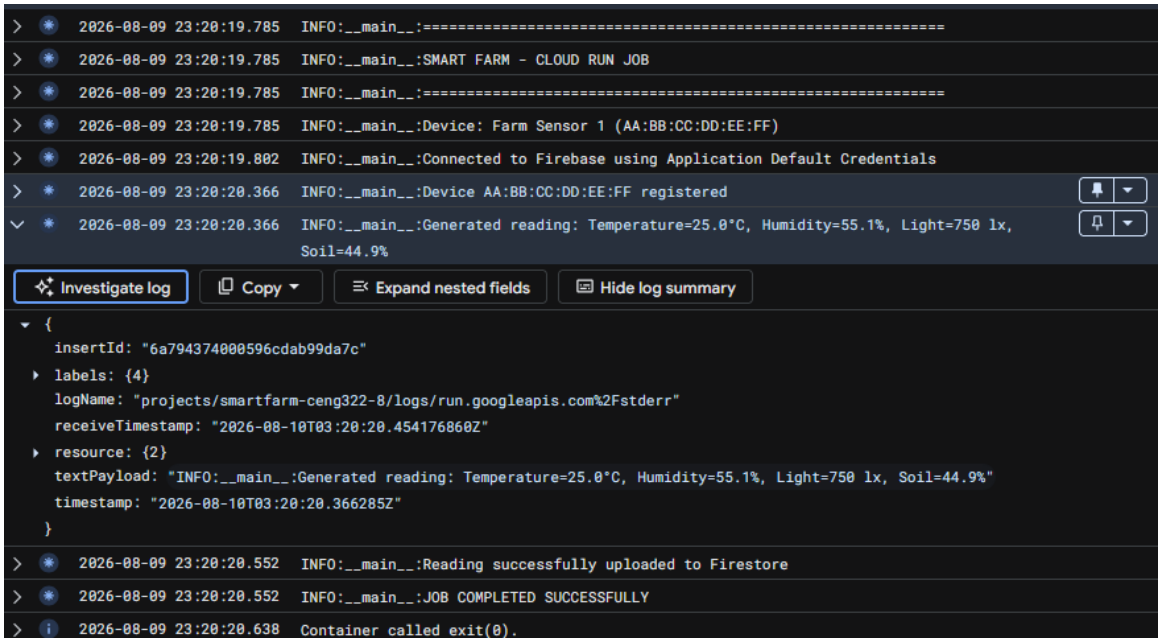
Anomalies (10% chance)

Randomly triggers one of:

- **High Temp**: 38-45°C
- **Low Soil Moisture**: 5-15%

Write Operations (per execution)

1. **Register Device** (first run only):
    - Creates/updates devices/{DEVICE_MAC}
    - Creates device_controls/{DEVICE_MAC}
    - Initializes sensor_data/{DEVICE_MAC}
2. **Generate Reading**:
    - Creates document in sensor_data/{DEVICE_MAC}/readings/{timestamp}
3. **Update Metadata**:
    - Updates lastUpdated in sensor_data/{DEVICE_MAC}
    - Increments totalReadings counter



Suggestions to the instructor for future projects, things you liked, and things you suggest being done differently and how.

What we liked: The sprint-based deliverable structure gave clear checkpoints and forced us to confront testing issues (Firebase init, Robolectric SDK mismatches) early rather than at final submission. Requiring real JUnit/Robolectric/Espresso coverage, not just optional testing, was valuable.

What we'd change:

1. **Teach branch/merge strategy earlier.** We didn't consistently use feature branches, causing merge conflicts in `build.gradle`/`libs.versions.toml` that cost real time to resolve.
2. **Provide an AndroidX Test compatibility reference.** Version mismatches between `espresso-core`, `androidx.test:monitor`, and `fragment-testing` caused confusing runtime errors (`NoSuchMethodError`, `NoClassDefFoundError`) unrelated to our actual test logic — a known compatibility cheat-sheet would save teams significant debugging time.
3. **Clarify tracking-tool expectations upfront.** We carried a Gantt chart responsibility from an earlier deliverable that wasn't clearly replaced by the later Scrum-only requirement. Stating one consistent tracking format at term start would avoid confusion.
4. **Add a UX/UI checkpoint earlier in the course.** Miscommunication over UI direction cost us roughly three weeks. A brief sign-off on mockups around Deliverable 2–3 could catch disagreements before implementation begins.